

UNIVERSIDAD DE PALERMO · DEPARTAMENTO DE INGENIERÍA

ARQUITECTURA WEB

Documentación de la API

Sistema de gestión de un taller mecánico



Actividad	Individual — Módulo 2
Alumno	Cristian Albuja
Legajo	0144050
Profesor	Ing. Diego Marafetti
Endpoints documentados	22

Todos los códigos de estado declarados en este documento fueron verificados ejecutando un request real contra la implementación.

1. Descripción general del backend

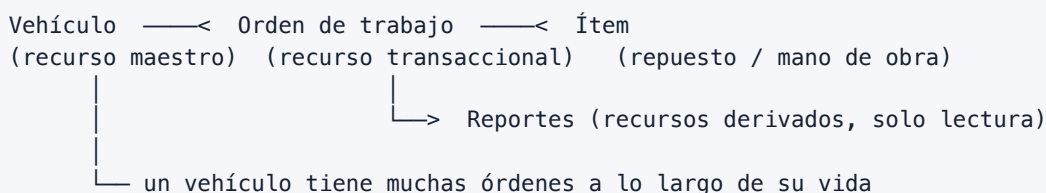
1.1. Dominio y problema que resuelve

El backend implementa la gestión operativa de un **taller mecánico**. El problema que resuelve es el seguimiento del trabajo que el taller realiza sobre las unidades de sus clientes: qué vehículos atiende, qué reparaciones se les hicieron, qué repuestos y horas de mano de obra consumió cada reparación, cuánto se facturó y qué unidades están pendientes de un nuevo service.

Antes de un sistema así, esa información vive en papel o en una planilla: no hay forma confiable de saber cuánto costó mantener una unidad a lo largo del tiempo, ni de detectar qué clientes deberían volver al taller. El backend expone esos datos como recursos consultables y modificables mediante una API RESTful.

1.2. Modelo de recursos

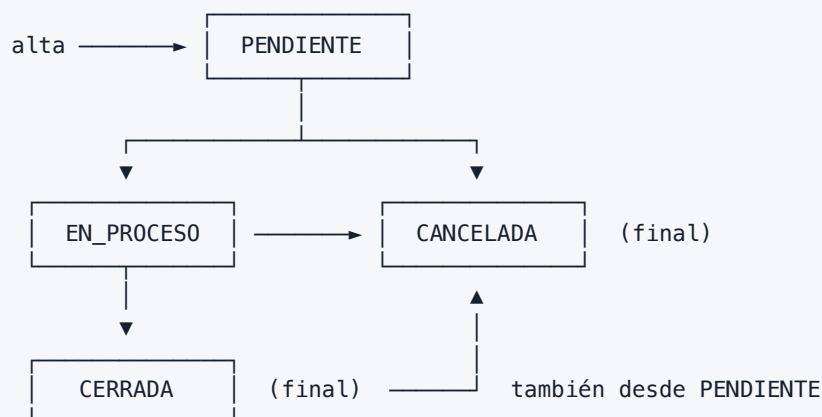
El dominio se estructura en dos recursos principales y un sub-recurso:



- **Vehículo** — la unidad que atiende el taller, identificada por su patente. Es el recurso maestro y sobre él se implementa el ABM completo.
- **Orden de trabajo** — el ingreso de un vehículo al taller por un trabajo determinado. Tiene un ciclo de vida propio y es el recurso que alimenta los reportes.
- **Ítem** — cada repuesto colocado o cada tarea de mano de obra dentro de una orden. No existe fuera de su orden, por eso se modela como sub-recurso anidado.
- **Reportes** — recursos derivados que se calculan a demanda a partir de los anteriores. No se almacenan y solo admiten lectura.

1.3. Ciclo de vida de una orden de trabajo

Varios de los códigos `409 Conflict` documentados más adelante se explican por esta máquina de estados. Cualquier transición que no figure en el diagrama es rechazada:



Dos reglas asociadas al cierre condicionan el contrato:

- Una orden **no puede cerrarse sin ítems cargados**: no se factura una orden vacía. El intento devuelve **409**.
- Al cerrarse, si el kilometraje de ingreso de la orden es mayor al registrado en el vehículo, **el kilometraje del vehículo se actualiza**. Es el dato más fresco que tiene el taller sobre esa unidad.

1.4. Datos por defecto

Al inicializarse, la aplicación carga automáticamente un set de datos de prueba: 8 vehículos (7 activos y 1 dado de baja) y 10 órdenes de trabajo distribuidas entre los cuatro estados, con sus ítems. Todos los ejemplos de este documento están tomados de ese set.

2. Convenciones del contrato

Las reglas de esta sección aplican a **todos** los endpoints. Documentarlas una sola vez acá evita repetirlas veintidós veces en la sección siguiente.

2.1. URL base y versionado

```
http://localhost:3001/api/v1
```

La versión viaja en la URI. Adoptar una estrategia de versionado desde el primer release, aunque sea la más simple, evita quedar atado a un contrato implícito de versión única cuando ya existen consumidores. Todas las rutas de este documento son relativas a esa base.

2.2. Diseño de las URIs

- Los recursos se nombran con **sustantivos en plural** (`/vehiculos` , `/ordenes`). La acción la aporta el verbo HTTP, nunca la URI: no existe `/crearVehiculo` .
- Los **filtros viajan por query string** y no como rutas distintas: `/vehiculos?marca=Toyota` es una vista de la colección, no un recurso nuevo.
- El **anidamiento se reserva para relaciones de pertenencia** (`/ordenes/{id}/items`) y no supera los dos niveles.

2.3. Content-Type

Todo request con cuerpo (`POST` , `PUT` , `PATCH`) debe enviar `Content-Type: application/json` . La API responde siempre `application/json` .

2.4. Formato de las respuestas de colección

Todos los listados devuelven la misma estructura, con los datos separados de la metadata de paginación:

```
{
  "datos": [ ... ],
  "meta": {
    "total": 8, "page": 1, "limit": 20,
    "totalPaginas": 1, "hayPaginaSiguiente": false
  }
}
```

Los parámetros de paginación son comunes a todas las colecciones: `page` (entero ≥ 1 , por defecto `1`) y `limit` (entero entre 1 y 100, por defecto `20`). El ordenamiento usa `sort=campo` para ascendente y `sort=-campo` para descendente.

2.5. Formato de los errores

Todos los errores, sin importar el código de estado, comparten una única forma. Esto permite que el cliente los maneje de manera genérica en lugar de tener un parser por endpoint:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "La solicitud contiene datos invalidos",
    "details": [
      { "campo": "patente", "mensaje": "'patente' no tiene un formato valido (AAA111 o AB123CD)" },
      { "campo": "anio", "mensaje": "'anio' debe estar entre 1950 y 2027" }
    ]
  }
}
```

El campo `code` es un identificador estable, pensado para que el cliente ramifique sobre él sin depender del texto del mensaje. El arreglo `details` trae el detalle campo por campo en los errores de validación y viene vacío en el resto.

CÓDIGO HTTP	ERROR.CODE	CUÁNDO LO DEVUELVE
400	VALIDATION_ERROR	Datos del cuerpo, parámetros de consulta o campos desconocidos inválidos
400	INVALID_JSON	El cuerpo del request no es JSON parseable
404	NOT_FOUND	El recurso identificado en la URI no existe
404	ROUTE_NOT_FOUND	La ruta solicitada no corresponde a ningún endpoint de la API
405	METHOD_NOT_ALLOWED	La URI existe pero no admite ese verbo. Incluye el header <code>Allow</code>
409	CONFLICT	El request es válido pero choca con el estado actual del sistema
413	PAYLOAD_TOO_LARGE	El cuerpo del request supera los 100 kB
415	UNSUPPORTED_MEDIA_TYPE	El <code>Content-Type</code> no es <code>application/json</code>
500	INTERNAL_SERVER_ERROR	Error no previsto. El detalle se registra en el servidor y no se envía al cliente

Criterio 400 frente a 409. Un 400 indica que el request está mal formado y el cliente puede corregirlo mirando solamente lo que envió. Un 409 indica que el request es correcto pero incompatible con el estado actual del servidor: la patente ya existe, la orden ya está cerrada, el vehículo tiene trabajo pendiente. El cliente no podía saberlo de antemano.

2.6. Comportamientos transversales

Estos tres códigos son alcanzables en toda la API y por eso no se repiten en cada ficha:

- **405 Method Not Allowed** — cualquier URI existente invocada con un verbo que no soporta. La respuesta incluye el header `Allow` con los verbos admitidos, tal como exige la especificación. Cada ficha de la sección 3 declara el `Allow` de su URI.
- **413 Payload Too Large** — todo endpoint que acepta cuerpo, si este supera los 100 kB.
- **500 Internal Server Error** — cualquier endpoint, ante un error no previsto. No es parte del comportamiento esperado.

2.7. Propiedades de los métodos

Cada ficha declara si el endpoint es *safe* e *idempotente*. Un método es **safe** cuando su invocación no le pide al servidor ningún cambio en los recursos que expone, e **idempotente** cuando invocarlo varias veces deja el sistema en el mismo estado que invocarlo una sola vez.

Sobre la idempotencia de DELETE. Repetir un DELETE sobre un recurso ya eliminado devuelve 404. Eso no contradice la idempotencia: la propiedad se refiere al *estado del sistema* tras completarse el request, no al código de estado devuelto. El recurso queda igualmente eliminado. El mismo razonamiento aplica a PUT /ordenes/{id}/estado, que devuelve 409 al repetirse pero deja la orden en el mismo estado.

Sobre PATCH. PATCH no es idempotente en general, porque nada impide que una implementación aplique incrementos. En esta API sí lo es: todos los campos modificables se asignan por valor absoluto, nunca por delta.

2.8. Nivel de madurez

NIVEL	REQUISITO	CUMPLIMIENTO
0	HTTP como mecanismo de transporte	Cumplido
1	Recursos identificados por URI	Cumplido — /vehiculos, /vehiculos/{id}, /ordenes/{id}/items
2	Uso semántico de verbos HTTP y códigos de estado	Cumplido — cinco verbos según su semántica, Location en los 201, Allow en los 405
3	Controles hipermedia (HATEOAS)	Parcial — el índice GET / publica las URIs de todos los recursos, pero las representaciones individuales no incluyen enlaces de navegación

La API se sitúa por lo tanto en el **Nivel 2 del Modelo de Madurez de Richardson**.

3. Documentación de los endpoints

Se documentan los **22 endpoints** que el backend expone efectivamente. Para cada uno se indican la ruta completa, el verbo HTTP, su propósito, sus propiedades, los parámetros que acepta, el cuerpo esperado cuando corresponde y todos los códigos de estado que puede devolver, con el caso concreto en que ocurre cada uno.

3.1. Recursos de servicio

01 **GET** `/api/v1`

Índice de la API. Devuelve la lista de recursos disponibles con su URI, de modo que un cliente que llega a la raíz pueda descubrir la API sin leer la documentación.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, OPTIONS** · Sin parámetros

CÓDIGO	CASO EN QUE OCURRE
200 OK	Índice devuelto correctamente. Es el único resultado posible: el endpoint no recibe entrada

02 **GET** `/api/v1/health`

Chequeo de salud. Confirma que el servidor está operativo e informa cuántos vehículos y órdenes cargó el set de datos por defecto, lo que permite verificar de un vistazo que la inicialización funcionó.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, OPTIONS** · Sin parámetros

CÓDIGO	CASO EN QUE OCURRE
200 OK	El servicio responde. Un servidor caído no devuelve este endpoint en absoluto

3.2. Vehículos

Lista la colección de vehículos del taller, con búsqueda, filtros, ordenamiento y paginación.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, POST, OPTIONS**

PARÁMETROS DE CONSULTA

PARÁMETRO	TIPO	DESCRIPCIÓN
q	texto	Búsqueda libre sobre patente, marca, modelo y titular
marca	texto	Filtra por marca exacta, sin distinguir mayúsculas
activo	booleano	Filtra por estado de alta
sort	texto	patente, marca, modelo, anio, kilometraje, id. Prefijo - para descendente. Por defecto patente
page	entero	Página, desde 1. Por defecto 1
limit	entero	Resultados por página, entre 1 y 100. Por defecto 20

CÓDIGO	CASO EN QUE OCURRE
200 OK	Listado devuelto, con los datos y la metadata de paginación. Una búsqueda sin resultados también devuelve 200, con datos vacío
400 Bad Request	page menor a 1, limit fuera del rango 1-100, o sort sobre un campo no admitido

Da de alta un vehículo nuevo. La patente es la clave natural del recurso y debe ser única en todo el sistema. La respuesta incluye el header `Location` con la URI del recurso creado.

Safe: **no** · Idempotente: **no** — dos invocaciones idénticas crearían dos vehículos si la patente no fuese única · Allow: **GET, POST, OPTIONS**

CUERPO ESPERADO

```
{
  "patente": "AZ999ZZ",
  "marca": "Honda",
  "modelo": "Civic",
  "anio": 2020,
  "kilometraje": 54000,
  "titularNombre": "Carla Nunez",
  "titularTelefono": "11-1234-5678",
  "activo": true
}
```

CAMPO	OBLIGATORIO	REGLAS
patente	Sí	Formato argentino: <code>AAA111</code> o <code>AB123CD</code> . Se normaliza a mayúsculas
marca	Sí	Texto de 2 a 40 caracteres
modelo	Sí	Texto de 1 a 40 caracteres
anio	Sí	Entero entre 1950 y el año próximo
kilometraje	Sí	Entero entre 0 y 2.000.000
titularNombre	Sí	Texto de 3 a 80 caracteres
titularTelefono	Sí	Texto de 6 a 30 caracteres
activo	No	Booleano. Por defecto <code>true</code>

CÓDIGO	CASO EN QUE OCURRE
201 Created	Vehículo creado. Devuelve la representación completa y el header <code>Location</code>
400 Bad Request	Falta un campo obligatorio, un valor no cumple sus reglas, se envió un campo que el recurso no acepta, o el cuerpo no es JSON parseable
409 Conflict	Ya existe otro vehículo con esa patente
413 Payload Too Large	El cuerpo supera los 100 kB
415 Unsupported Media Type	El <code>Content-Type</code> no es <code>application/json</code>

05

GET

`/api/v1/vehiculos/{id}`

Devuelve la representación completa de un vehículo identificado por su id.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, PUT, PATCH, DELETE, OPTIONS**

PARÁMETRO DE RUTA

`id` — identificador numérico del vehículo.

CÓDIGO	CASO EN QUE OCURRE
200 OK	Vehículo encontrado
404 Not Found	No existe un vehículo con ese id. También se devuelve cuando el id no es numérico, porque en ese caso tampoco identifica ningún recurso

06

PUT

`/api/v1/vehiculos/{id}`

Reemplaza por completo la representación del vehículo. El cliente debe enviar **todos** los campos obligatorios: se validan con las mismas reglas que el alta. Es la diferencia esencial con `PATCH`.

Safe: **no** · Idempotente: **sí** — repetir el mismo request deja el vehículo en idéntico estado · Allow: **GET, PUT, PATCH, DELETE, OPTIONS**

CUERPO ESPERADO

```
{
  "patente": "AB123CD",
  "marca": "Toyota",
  "modelo": "Corolla XEI",
  "anio": 2019,
  "kilometraje": 90000,
  "titularNombre": "Laura Gimenez",
  "titularTelefono": "11-4455-8890",
  "activo": true
}
```

CÓDIGO	CASO EN QUE OCURRE
200 OK	Vehículo reemplazado. Devuelve la representación resultante
400 Bad Request	Falta algún campo obligatorio, un valor es inválido, o se envió un campo desconocido
404 Not Found	No existe un vehículo con ese id
409 Conflict	La patente enviada ya pertenece a otro vehículo, o el kilometraje es menor al registrado (un vehículo no desanda kilómetros)
413 Payload Too Large	El cuerpo supera los 100 kB
415 Unsupported Media Type	El <code>Content-Type</code> no es <code>application/json</code>

07

PATCH

`/api/v1/vehiculos/{id}`

Modifica parcialmente un vehículo: aplica únicamente los campos enviados y conserva el resto. Es la vía recomendada para actualizar un solo dato, como el kilometraje tras un service, o para dar de baja lógicamente una unidad.

Safe: **no** · Idempotente: **sí** en esta API — los campos se asignan por valor absoluto · Allow: **GET, PUT, PATCH, DELETE, OPTIONS**

CUERPO ESPERADO

Cualquier subconjunto no vacío de los campos del alta:

```
{ "kilometraje": 92000 }  
  
{ "activo": false }  
  
{ "titularNombre": "Laura Gimenez", "titularTelefono": "11-5555-0000" }
```

CÓDIGO	CASO EN QUE OCURRE
200 OK	Vehículo modificado. Devuelve la representación completa resultante
400 Bad Request	El cuerpo viene vacío (un PATCH sin cambios no tiene sentido), algún valor enviado es inválido, o se envió un campo desconocido
404 Not Found	No existe un vehículo con ese id
409 Conflict	La patente enviada ya pertenece a otro vehículo, o el kilometraje es menor al registrado
413 Payload Too Large	El cuerpo supera los 100 kB
415 Unsupported Media Type	El Content-Type no es application/json

08

DELETE

`/api/v1/vehiculos/{id}`

Elimina un vehículo junto con sus órdenes históricas. La baja se bloquea si la unidad tiene órdenes abiertas, porque eliminarla dejaría órdenes apuntando a un vehículo inexistente. En ese caso el mensaje de error sugiere la baja lógica como alternativa.

Safe: **no** · Idempotente: **sí** — repetir el request deja el vehículo igualmente eliminado, aunque la segunda invocación devuelva 404 · Allow: **GET, PUT, PATCH, DELETE, OPTIONS** · Sin cuerpo

CÓDIGO	CASO EN QUE OCURRE
204 No Content	Vehículo eliminado. No se devuelve cuerpo: la baja no tiene nada útil que informar
404 Not Found	No existe un vehículo con ese id
409 Conflict	El vehículo tiene al menos una orden en estado PENDIENTE o EN_PROCESO

09

GET

`/api/v1/vehiculos/{id}/ordenes`

Devuelve el historial completo de órdenes de trabajo de un vehículo, ordenado de la más reciente a la más antigua. Es un sub-recurso: las órdenes pertenecen al vehículo y la URI lo expresa por anidamiento.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, OPTIONS** · Sin parámetros de consulta

CÓDIGO	CASO EN QUE OCURRE
200 OK	Historial devuelto. Un vehículo sin órdenes devuelve 200 con la lista vacía
404 Not Found	El vehículo indicado en la URI no existe

3.3. Órdenes de trabajo

10

GET

`/api/v1/ordenes`

Lista la colección de órdenes de trabajo con filtros, ordenamiento y paginación. Cada orden se devuelve enriquecida con los datos básicos de su vehículo, para que una grilla no necesite hacer un request adicional por cada fila.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, POST, OPTIONS**

PARÁMETROS DE CONSULTA

PARÁMETRO	TIPO	DESCRIPCIÓN
estado	texto	Uno o varios estados separados por coma, ej. <code>PENDIENTE, EN_PROCESO</code>
vehiculoId	entero	Filtra por vehículo
mecanico	texto	Coincidencia parcial sobre el nombre del mecánico
desde	fecha	Límite inferior del rango sobre la fecha de ingreso, en formato ISO 8601
hasta	fecha	Límite superior del rango sobre la fecha de ingreso, en formato ISO 8601
sort	texto	<code>id</code> , <code>numero</code> , <code>fechaIngreso</code> , <code>estado</code> , <code>total</code> , <code>mecanico</code> . Por defecto <code>-fechaIngreso</code>
page	entero	Página, desde 1. Por defecto <code>1</code>
limit	entero	Resultados por página, entre 1 y 100. Por defecto <code>20</code>

CÓDIGO	CASO EN QUE OCURRE
200 OK	Listado devuelto con los datos y la metadata de paginación
400 Bad Request	<code>page</code> o <code>limit</code> fuera de rango, <code>sort</code> sobre un campo no admitido, o <code>desde</code> / <code>hasta</code> con una fecha no interpretable

11

POST

`/api/v1/ordenes`

Abre una orden de trabajo sobre un vehículo activo. Toda orden nace en estado `PENDIENTE` y sin ítems: el estado no se acepta desde el cliente, solo puede avanzar por el endpoint de transición. Permitir fijarlo en el alta habilitaría crear órdenes ya cerradas, saltando la validación de que una orden cerrada necesita ítems.

Safe: **no** · Idempotente: **no** — repetir el request abre una segunda orden · Allow: **GET, POST, OPTIONS**

CUERPO ESPERADO

```
{
  "vehiculoId": 1,
  "descripcion": "Cambio de correa de distribucion y tensor",
  "mecanico": "Ruben Paz",
  "kilometrajeIngreso": 95000
}
```

CAMPO	OBLIGATORIO	REGLAS
vehiculoId	Sí	Entero. El vehículo debe existir y estar activo
descripcion	Sí	Texto de 5 a 300 caracteres
mecanico	Sí	Texto de 3 a 80 caracteres
kilometrajeIngreso	Sí	Entero. No puede ser menor al kilometraje actual del vehículo

CÓDIGO	CASO EN QUE OCURRE
201 Created	Orden creada en estado <code>PENDIENTE</code> , con su número de negocio asignado y el header <code>Location</code>
400 Bad Request	Falta un campo obligatorio, un valor es inválido, se envió un campo desconocido, o el vehículo referenciado no existe . Este último caso es 400 y no 404 porque el recurso pedido (<code>/ordenes</code>) sí existe: lo que está mal es un dato del cuerpo
409 Conflict	El vehículo está dado de baja y no admite nuevas órdenes, o el kilometraje de ingreso es menor al último registrado
413 Payload Too Large	El cuerpo supera los 100 kB
415 Unsupported Media Type	El <code>Content-Type</code> no es <code>application/json</code>

12

GET

`/api/v1/ordenes/{id}`

Devuelve una orden de trabajo con todos sus ítems y el total calculado a partir de ellos. El total nunca se almacena: se deriva de los ítems en cada lectura, de modo que no pueda quedar desincronizado.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, PATCH, DELETE, OPTIONS**

CÓDIGO	CASO EN QUE OCURRE
200 OK	Orden encontrada
404 Not Found	No existe una orden con ese id

13

PATCH

`/api/v1/ordenes/{id}`

Modifica la descripción del trabajo y/o el mecánico asignado. Solo se admite mientras la orden esté abierta.

Este recurso **no expone PUT** deliberadamente. Una orden tiene campos que el cliente no puede fijar (numero, estado, fechas, total), y ofrecer un reemplazo completo daría a entender que se pueden enviar todos.

Safe: **no** · Idempotente: **sí** en esta API · Allow: **GET, PATCH, DELETE, OPTIONS**

CUERPO ESPERADO

```
{ "descripcion": "Revision general del motor", "mecanico": "Nadia Ocampo" }
```

Ambos campos son opcionales, pero debe enviarse al menos uno. `descripcion` admite de 5 a 300 caracteres y `mecanico` de 3 a 80.

CÓDIGO	CASO EN QUE OCURRE
200 OK	Orden modificada
400 Bad Request	El cuerpo viene vacío, un valor es inválido, o se envió un campo que este endpoint no acepta (por ejemplo <code>estado</code>)
404 Not Found	No existe una orden con ese id
409 Conflict	La orden está <code>CERRADA</code> o <code>CANCELADA</code> : es un documento cerrado y no se reescribe
413 Payload Too Large	El cuerpo supera los 100 kB
415 Unsupported Media Type	El <code>Content-Type</code> no es <code>application/json</code>

14

DELETE

`/api/v1/ordenes/{id}`

Elimina una orden abierta o cancelada. Las órdenes cerradas no se eliminan: forman el historial facturado que alimenta los reportes. Para descartar una orden sin borrarla existe el estado `CANCELADA` .

Safe: **no** · Idempotente: **sí** · Allow: **GET, PATCH, DELETE, OPTIONS** · Sin cuerpo

CÓDIGO	CASO EN QUE OCURRE
204 No Content	Orden eliminada. Sin cuerpo de respuesta
404 Not Found	No existe una orden con ese id
409 Conflict	La orden está <code>CERRADA</code> y forma parte del historial facturado

15

PUT

`/api/v1/ordenes/{id}/estado`

Aplica una transición de estado respetando la máquina de estados de la sección 1.3. Al pasar a `CERRADA`, además, actualiza el kilometraje del vehículo si el de ingreso es mayor.

El estado se modela como **sub-recurso propio** y no como un campo más del PATCH porque cambiarlo dispara efectos de negocio —validar la transición, exigir ítems para cerrar, actualizar el vehículo— que no son una simple edición de un atributo. Se usa PUT y no POST porque la operación fija el estado a un valor determinado, lo que la hace idempotente en su intención.

Safe: **no** · Idempotente: **sí** — repetir el request deja la orden en el mismo estado, aunque la segunda invocación devuelva 409 · Allow: **PUT, OPTIONS**

CUERPO ESPERADO

```
{ "estado": "EN_PROCESO" }
```

Valores admitidos: `PENDIENTE`, `EN_PROCESO`, `CERRADA`, `CANCELADA`. Es el único campo aceptado.

CÓDIGO	CASO EN QUE OCURRE
200 OK	Transición aplicada. Devuelve la orden con su nuevo estado y, si corresponde, la fecha de cierre
400 Bad Request	El valor de <code>estado</code> no pertenece al conjunto admitido, o se envió otro campo
404 Not Found	No existe una orden con ese id
409 Conflict	Tres casos: la transición no está permitida por la máquina de estados; la orden ya se encuentra en el estado solicitado; o se intenta cerrar una orden sin ítems cargados
413 Payload Too Large	El cuerpo supera los 100 kB
415 Unsupported Media Type	El <code>Content-Type</code> no es <code>application/json</code>

3.4. Ítems de una orden

16

GET

`/api/v1/ordenes/{id}/items`

Lista los repuestos y las tareas de mano de obra cargados en una orden, con el subtotal de cada uno.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, POST, OPTIONS**

CÓDIGO	CASO EN QUE OCURRE
200 OK	Ítems devueltos. Una orden sin ítems devuelve 200 con la lista vacía
404 Not Found	La orden indicada en la URI no existe

Agrega un repuesto o una tarea de mano de obra a una orden abierta. El subtotal del ítem y el total de la orden se recalculan automáticamente.

Safe: **no** · Idempotente: **no** — repetir el request agrega un segundo ítem · Allow: **GET, POST, OPTIONS**

CUERPO ESPERADO

```
{
  "tipo": "REPUESTO",
  "descripcion": "Kit de distribucion",
  "cantidad": 1,
  "precioUnitario": 195000
}
```

CAMPO	OBLIGATORIO	REGLAS
tipo	Sí	REPUESTO o MANO_DE_OBRA
descripcion	Sí	Texto de 3 a 150 caracteres
cantidad	Sí	Entero entre 1 y 999
precioUnitario	Sí	Número mayor o igual a 0

CÓDIGO	CASO EN QUE OCURRE
201 Created	Ítem agregado. Devuelve el ítem con su subtotal calculado y el header <code>Location</code> . El id del ítem es único dentro de su orden, no globalmente
400 Bad Request	Falta un campo, el tipo no pertenece al conjunto admitido, la cantidad no es un entero positivo, o se envió un campo desconocido
404 Not Found	La orden indicada en la URI no existe
409 Conflict	La orden está <code>CERRADA</code> o <code>CANCELADA</code> y ya no admite carga de ítems
413 Payload Too Large	El cuerpo supera los 100 kB
415 Unsupported Media Type	El <code>Content-Type</code> no es <code>application/json</code>

18 **DELETE** /api/v1/ordenes/{id}/items/{itemId}

Quita un ítem de una orden abierta y recalcula el total.

Safe: **no** · Idempotente: **sí** · Allow: **DELETE, OPTIONS** · Sin cuerpo

PARÁMETROS DE RUTA

`id` — identificador de la orden. `itemId` — identificador del ítem dentro de esa orden.

CÓDIGO	CASO EN QUE OCURRE
204 No Content	Ítem quitado. Sin cuerpo de respuesta
404 Not Found	La orden no existe, o el ítem no existe dentro de esa orden
409 Conflict	La orden está <code>CERRADA</code> o <code>CANCELADA</code>

3.5. Reportes

Los cuatro reportes son recursos derivados y de solo lectura: se calculan a demanda a partir de vehículos y órdenes, y no se almacenan. Por eso responden siempre con `Cache-Control: no-store` y solo admiten `GET`. Criterio común: únicamente las órdenes `CERRADA` cuentan como facturación, porque una orden abierta todavía puede cambiar de monto y una cancelada nunca se cobró.

19 **GET** /api/v1/reportes/resumen

Indicadores generales del taller: facturación total, ticket promedio por orden, cantidad de órdenes discriminadas por estado y conteo de vehículos activos e inactivos.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, OPTIONS** · Sin parámetros

CÓDIGO	CASO EN QUE OCURRE
200 OK	Reporte calculado. Es el único resultado posible: el endpoint no recibe entrada

20 **GET** /api/v1/reportes/facturacion-mensual

Facturación agrupada por mes. Devuelve siempre los doce meses, incluidos los que no tuvieron movimiento: un gráfico con meses faltantes distorsiona la lectura de la tendencia.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, OPTIONS**

PARÁMETRO DE CONSULTA

`anio` — entero entre 2000 y el año próximo. Por defecto, el año en curso.

CÓDIGO	CASO EN QUE OCURRE
200 OK	Reporte calculado. Un año sin actividad devuelve la grilla completa en cero
400 Bad Request	<code>anio</code> no es un entero, o está fuera del rango admitido

21

GET

/api/v1/reportes/costos-por-vehiculo

Costo histórico de mantenimiento de cada unidad, discriminado entre repuestos y mano de obra, ordenado de mayor a menor facturación.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, OPTIONS**

PARÁMETRO DE CONSULTA

`limit` — entero entre 1 y 100, para pedir el top N. Por defecto se devuelven todos los vehículos.

CÓDIGO	CASO EN QUE OCURRE
200 OK	Reporte calculado
400 Bad Request	<code>limit</code> no es un entero, o está fuera del rango admitido

22

GET

/api/v1/reportes/service-vencido

Vehículos activos que recorrieron más kilómetros que el umbral desde su última orden cerrada. Una unidad que nunca ingresó al taller se mide contra su kilometraje total y se marca con `nuncaIngreso`. El reporte incluye el teléfono del titular, porque su caso de uso es llamarlo para ofrecerle el turno.

Safe: **sí** · Idempotente: **sí** · Allow: **GET, OPTIONS**

PARÁMETRO DE CONSULTA

`umbralKm` — entero entre 1.000 y 100.000. Por defecto `10000`.

CÓDIGO	CASO EN QUE OCURRE
200 OK	Reporte calculado. Si ninguna unidad supera el umbral, devuelve la lista vacía
400 Bad Request	<code>umbralKm</code> no es un entero, o está fuera del rango admitido

4. Tabla resumen

Vista consolidada de los 22 endpoints. Los códigos 405 y 500, por ser transversales a toda la API, no se repiten en cada fila (ver sección 2.6).

#	VERBO	ROUTA	PROPÓSITO	CÓDIGOS DE ESTADO
01	GET	/api/v1	Índice de recursos	200
02	GET	/api/v1/health	Chequeo de salud	200
03	GET	/api/v1/vehiculos	Listar vehículos	200, 400
04	POST	/api/v1/vehiculos	Dar de alta un vehículo	201, 400, 409, 413, 415
05	GET	/api/v1/vehiculos/{id}	Obtener un vehículo	200, 404
06	PUT	/api/v1/vehiculos/{id}	Reemplazar un vehículo	200, 400, 404, 409, 413, 415
07	PATCH	/api/v1/vehiculos/{id}	Modificar parcialmente un vehículo	200, 400, 404, 409, 413, 415
08	DELETE	/api/v1/vehiculos/{id}	Eliminar un vehículo	204, 404, 409
09	GET	/api/v1/vehiculos/{id}/ordenes	Historial de órdenes del vehículo	200, 404
10	GET	/api/v1/ordenes	Listar órdenes de trabajo	200, 400
11	POST	/api/v1/ordenes	Abrir una orden de trabajo	201, 400, 409, 413, 415
12	GET	/api/v1/ordenes/{id}	Obtener una orden con sus ítems	200, 404
13	PATCH	/api/v1/ordenes/{id}	Modificar una orden abierta	200, 400, 404, 409, 413, 415
14	DELETE	/api/v1/ordenes/{id}	Eliminar una orden	204, 404, 409
15	PUT	/api/v1/ordenes/{id}/estado	Cambiar el estado de una orden	200, 400, 404, 409, 413, 415
16	GET	/api/v1/ordenes/{id}/items	Listar los ítems de una orden	200, 404
17	POST	/api/v1/ordenes/{id}/items	Agregar un ítem a una orden	201, 400, 404, 409, 413, 415
18	DELETE	/api/v1/ordenes/{id}/items/{itemId}	Quitar un ítem de una orden	204, 404, 409
19	GET	/api/v1/reportes/resumen	Indicadores generales	200
20	GET	/api/v1/reportes/facturacion-mensual	Facturación por mes	200, 400
21	GET	/api/v1/reportes/costos-por-vehiculo	Costo de mantenimiento por unidad	200, 400

#	VERBO	RUTA	PROPÓSITO	CÓDIGOS DE ESTADO
22	GET	/api/v1/reportes/service-vencido	Unidades con el service vencido	200, 400

4.1. Correspondencia entre operación, verbo y código de éxito

El contrato sigue la correspondencia habitual entre la intención de la operación y el par verbo/código de estado, en lugar de devolver `200 OK` para todo:

OPERACIÓN	VERBO	CÓDIGO DE ÉXITO	ENDPOINTS
Obtener un recurso o una colección	GET	200 OK	01–03, 05, 09, 10, 12, 16, 19–22
Crear un recurso nuevo	POST	201 Created	04, 11, 17
Reemplazar un recurso completo	PUT	200 OK	06, 15
Modificar parcialmente un recurso	PATCH	200 OK	07, 13
Eliminar un recurso	DELETE	204 No Content	08, 14, 18

5. Verificación del contrato

Los códigos de estado de este documento no se listaron *únicamente* a partir de la lectura del código: **cada uno se comprobó ejecutándolo**. El procedimiento fue el siguiente:

1. Se recorrieron los archivos de rutas, controladores, servicios y middlewares para identificar qué códigos son alcanzables en cada endpoint.
2. Se construyó un caso de prueba por cada par (*endpoint*, *código de estado*), partiendo siempre del mismo set de datos inicial.
3. Se ejecutaron los 82 casos resultantes, comparando el código obtenido contra el declarado.

Ese contraste es el que le da valor al procedimiento: la primera corrida detectó **tres discrepancias** entre el contrato que se pretendía documentar y el comportamiento real. Las tres se corrigieron en la implementación:

ENDPOINT	CASO	DECLARADO	OBTENIDO	DIAGNÓSTICO Y CORRECCIÓN
GET /ordenes	?desde=xxx	400	500	<code>new Date('xxx')</code> no lanza ninguna excepción: devuelve una fecha inválida, que aparenta ser un valor utilizable. El error recién se producía más adelante, al invocar <code>toISOString()</code> sobre ella para comparar el rango, y como en ese punto nadie lo esperaba terminaba escalando a <code>500</code> . Se agregó la validación del parámetro <i>antes</i> de tocar los datos: ahora devuelve <code>400</code> indicando el campo
GET /ordenes	?hasta=xxx	400	500	El otro extremo del mismo filtro de rango, con idéntica causa y corrección
POST /api/v1	Verbo no soportado	405	404	El índice era el único recurso de la API que no declaraba sus verbos admitidos, por lo que respondía <code>404</code> donde todos los demás responden <code>405</code> . Se unificó el comportamiento

Por qué eran defectos y no decisiones de diseño. Un `500` le comunica al cliente que el servidor falló y que no hay nada que él pueda hacer al respecto. Pero una fecha mal escrita es un dato inválido que el cliente sí puede corregir, y eso pertenece al rango `4xx`: devolver `500` lo mandaba a buscar el problema en el lugar equivocado. El caso del `404` es más leve, pero rompía la uniformidad del contrato: un cliente que ya había aprendido que esta API responde `405` ante un verbo no admitido recibía otra cosa en un único endpoint.

Tras las correcciones, los **82 casos coinciden** con lo documentado.

5.1. Cómo se hizo repetible la verificación

Los 82 casos no quedaron como una comprobación hecha una sola vez: se incorporaron al código del backend como una suite de pruebas automatizadas (`tests/contrato-api.test.js`), ejecutable con `npm test`. Cada prueba afirma uno de los códigos de estado declarados en estas páginas, de modo que **si el comportamiento de un endpoint cambia, la suite falla y señala que esta documentación quedó desactualizada**.

Al pasarlos a suite se agregaron seis casos que la comprobación inicial no cubría —el `405` del health check, un rango de fechas válido, el `405` de los cuatro reportes y una verificación de que todos los errores comparten la misma forma —, con lo cual la suite de contrato contiene **88 pruebas**. Sumadas a las del resto del backend, el proyecto tiene **178 pruebas**.

Sobre el código del backend. La implementación que este documento describe —incluida la suite de verificación— se incorporará a este repositorio junto con el Trabajo Práctico Integrador, en la instancia correspondiente de la cursada. Esta entrega comprende la documentación del contrato.

5.2. Especificación OpenAPI

El mismo contrato está publicado además como una especificación **OpenAPI 3.0.3** en el archivo `openapi.yaml` que acompaña a este documento. Se puede abrir en `editor.swagger.io` para obtener documentación navegable y probar cada endpoint contra el servidor local. La especificación fue validada con el linter *Redocly* y no presenta errores.

5.3. Documentación en el repositorio

El contenido de este documento está también integrado, en formato Markdown, en el `README.md` de esta misma carpeta del repositorio.